
Scalable Hyperparameter-Divergent Ensemble Training with Automatic Learning Rate Exploration for Large Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Training large neural networks with data-parallel stochastic gradient descent al-
2 locates N GPU replicas to compute effectively identical updates—a practice that
3 leaves the rich space of learning rate configurations entirely unexplored during
4 training. We propose **Hyperparameter-Divergent Ensemble Training (HDET)**,
5 a method that repurposes these replicas for simultaneous learning rate exploration
6 at negligible communication overhead. HDET operates in alternating phases: a
7 *fan-out* stage in which replicas train independently under a structured, symmetric
8 spread of learning rates, and a *converge* stage in which parameters are averaged
9 across all replicas via AllReduce every T steps. Building on this ensemble sub-
10 strate, we further propose an **automatic learning rate (auto-LR) controller** that
11 treats the relative training loss across replicas as a performance signal, updating the
12 shared base schedule toward higher-performing configurations via a momentum-
13 based gradient-free meta-update. The combined method produces a self-adapting
14 learning rate schedule that improves both optimization quality and generalization
15 without additional hyperparameter sweeps or training budget.

16 Crucially, the framework generalizes beyond learning rate: any scalar hyperparam-
17 eter that does not alter model architecture—such as dropout rate, attention scale
18 temperature, or weight-decay coefficient—can be explored across replicas using
19 the same fan-out/converge protocol, with inter-replica loss differences serving
20 as zero-order *hypergradients* that guide the search direction. HDET is imple-
21 mented as a drop-in replacement for PyTorch’s OneCycleLR scheduler, requiring
22 no changes to model architecture, optimizer, or data pipeline. Code is available at
23 https://github.com/hailingc/ensemble_training.

1 Introduction

24 The learning rate schedule is among the most consequential hyperparameter choices in large-model
25 training. Popular schedules—one-cycle annealing [Smith and Topin, 2019], cosine decay [Loshchilov
26 and Hutter, 2017], and linear warmup-decay [Vaswani et al., 2017]—must be specified before training
27 begins, leaving practitioners to rely on expensive grid searches or heuristics calibrated on smaller
28 proxy models. Even when a good schedule is found, it may not remain optimal as model or dataset
29 scale changes across training runs.

30 Simultaneously, large-model training routinely parallelizes across hundreds of GPU replicas using
31 data-parallel stochastic gradient descent (DP-SGD). In the standard paradigm, every replica maintains
32 an identical copy of the model parameters and advances under an identical learning rate; the replica’s
33 sole distinguishing role is the mini-batch it processes. Beyond improving gradient estimates through a
34

larger effective batch, the additional hardware contributes *no qualitative diversity* to the optimization trajectory.

We observe that this uniformity is a missed opportunity. The N parallel replicas could explore N distinct learning rate trajectories simultaneously, at no additional hardware cost. Moreover, if those diverging trajectories periodically merge through parameter averaging, the resulting model can harvest ensemble-like generalization benefits within a *single* training run.

We formalize this intuition in **Hyperparameter-Divergent Ensemble Training (HDET)**, which interleaves two phases atop any existing DDP+OneCycleLR setup:

1. **Fan-out:** each of N GPU replicas trains under a distinct learning rate drawn from a symmetric spread $[\bar{\eta}(1 - \alpha), \bar{\eta}(1 + \alpha)]$ around the current base schedule $\bar{\eta}^{(t)}$, where $\alpha \geq 0$ is the spread ratio;
2. **Converge:** every T training steps, parameters are averaged across all replicas via AllReduce, producing a unified model that seeds the next fan-out round.

On top of this ensemble substrate we introduce an **auto-LR controller** that uses the relative training loss across replicas as a performance signal. Replicas with below-average loss receive proportionally higher weight in a softmax-weighted average of per-rank learning rates; the resulting performance deviation drives a momentum-based velocity that shifts the base schedule up or down at each sync point. If high-LR replicas consistently outperform low-LR ones, the base schedule adapts upward; if the opposite holds, it adapts downward. This constitutes a gradient-free, within-run optimizer over the learning rate schedule, eliminating the need for a priori schedule selection.

Contributions.

- We propose HDET, a training method that transforms existing DDP replicas into a structured learning rate exploration ensemble at zero hardware overhead (§3).
- We introduce an automatic LR controller that uses rank-wise loss signals to adaptively update the learning rate via a gradient-free momentum update, requiring no additional tuning (§3.4).
- We identify a practical two-stage strategy—warm noisy initialization from a pre-trained checkpoint followed by HDET ensemble training—that jointly maximizes model quality and training stability: warm initialization seeds each replica near a high-quality solution, while periodic weight averaging prevents divergence under aggressive learning rates that would destabilize either technique alone (§3.5, §4).
- We demonstrate empirically that HDET consistently improves final model quality and convergence speed on production-scale training tasks (§4).
- We release a reference implementation as a drop-in PyTorch scheduler at https://github.com/hailingc/ensemble_training.

2 Related Work

Learning-rate scheduling and online adaptation. Standard schedules—one-cycle [Smith and Topin, 2019], cyclical [Smith, 2017], cosine annealing [Loshchilov and Hutter, 2017], and warmup-decay [Vaswani et al., 2017]—must be fixed before training; OneCycleLR [Paszke et al., 2019] is the base scheduler HDET subclasses. Online methods eliminate this: Hypergradient Descent [Baydin et al., 2018], L4 [Rolinek and Martius, 2018], D-Adaptation [Defazio and Mishchenko, 2023], DoG [Ivgi et al., 2023], and Schedule-Free [Defazio et al., 2024] all derive step sizes from gradient statistics or loss dynamics. HDET differs: its adaptation signal is *cross-replica performance* rather than gradient information.

Learning-rate diversity and population-style exploration. Alrao [Blier et al., 2019] assigns different learning rates *within* a single network, covering a wide LR range at negligible cost. PBT [Jaderberg et al., 2017] maintains a population of models, periodically evaluating and copying hyperparameters from better to worse members. HDET shares concurrent hyperparameter exploration but reuses existing DDP replicas and merges them by AllReduce averaging rather than replacement, requiring no extra workers and preserving gradient information from every replica.

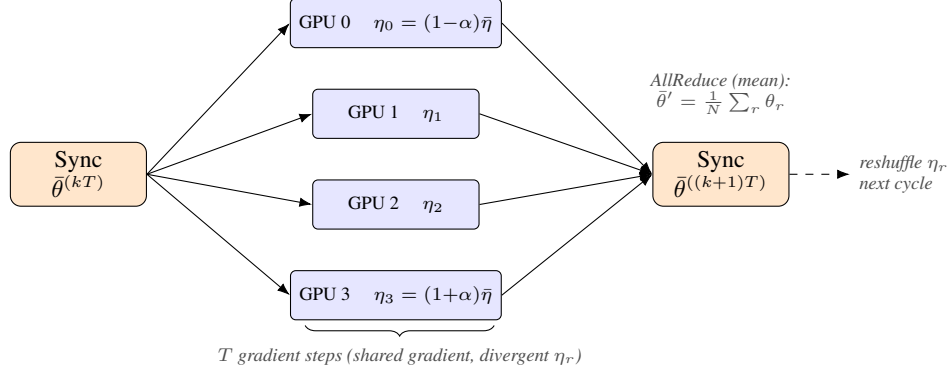


Figure 1: One HDET fan-out/converge cycle with $N=4$ replicas and spread ratio α . All replicas share the same all-reduced gradient at each step but take steps of different sizes, causing parameters to diverge. Every T steps an AllReduce average collapses the ensemble into a single model. Learning rates are randomly reassigned (reshuffled) before the next cycle.

85 **Distributed data-parallel training.** Standard DP-SGD [Li et al., 2020] synchronizes gradients via
 86 AllReduce, keeping all ranks at identical parameters $\theta_r^{(t)} = \theta^{(t)}$. HDET retains gradient synchroniza-
 87 tion but assigns rank-specific LR multipliers, letting parameters diverge between sync intervals.

88 **Weight averaging and ensemble merging.** SWA [Izmailov et al., 2018], SWAP [Gupta et al.,
 89 2020], EMA [Ho et al., 2020], and Model Soups [Wortsman et al., 2022] all average model weights
 90 to reach flatter minima or improve generalization, but do so sequentially or after training. HDET
 91 instead repeatedly averages N simultaneously diverging replicas *during* training—generating richer
 92 diversity at the same wall-clock time.

93 **Worker divergence and periodic synchronization.** EASGD [Zhang et al., 2015], Local SGD
 94 [Haddadpour et al., 2019], DiLoCo [Douillard et al., 2023], and GRAWA [Dimlioglu and Choro-
 95 manska, 2024] all allow workers to drift from a shared solution between sync events to reduce
 96 communication or improve convergence. HDET differs: divergence is *intentionally induced* by a
 97 structured LR spread while standard synchronized-gradient DDP is retained throughout.

98 **Gradient-free population optimization.** NES [Wierstra et al., 2014], ES [Salimans et al., 2017],
 99 Bayesian optimization [Snoek et al., 2012], and random search [Bergstra et al., 2011] optimize
 100 hyperparameters across separate runs or via population fitness signals. HDET’s auto-LR uses inter-
 101 replica loss differences as a lightweight within-run signal, requiring no restarts or separate evaluation
 102 infrastructure.

103 **Deep ensembles.** Deep Ensembles [Lakshminarayanan et al., 2017] capture epistemic uncertainty
 104 by training N independent models from different initializations, at $N \times$ cost; diversity arises primarily
 105 from models reaching different loss basins [Fort et al., 2019]. HDET generates comparable diversity
 106 within a single run by using the LR spread to push replicas toward different basins before averaging.

107 3 Hyperparameter-Divergent Ensemble Training

108 3.1 Overview

109 HDET adds three targeted modifications to standard DDP+OneCycleLR training: (1) a structured LR
 110 spread across replicas at initialization; (2) periodic all-reduce weight averaging every T steps; and
 111 (3) an optional auto-LR controller that adjusts the base schedule between sync intervals. Figure 1
 112 illustrates the full fan-out/converge cycle.

113 3.2 Learning Rate Divergence (Fan-Out)

114 For N ranks and spread ratio $\alpha \geq 0$, define $\delta = \max(\frac{N-1}{2}, 0.5)$ and assign each rank r the multiplier

$$\rho_r = 1 + \alpha \cdot \frac{r - \frac{N-1}{2}}{\delta}, \quad (1)$$

115 so that $\sum_r \rho_r = N$ (symmetric around 1) and $\rho_r \in [1-\alpha, 1+\alpha]$. Rank r trains under $\eta_r^{(t)} = \eta^{(t)} \cdot \rho_r$,
 116 where $\eta^{(t)}$ is the shared OneCycleLR base schedule, but all ranks share the same AllReduce gradient:

$$\theta_r^{(t+1)} = \theta_r^{(t)} - \eta_r^{(t)} \cdot \bar{g}^{(t)}. \quad (2)$$

117 Setting $\alpha=0$ recovers standard DDP exactly.

118 3.3 Weight Averaging (Converge)

119 Every T steps, HDET AllReduce-averages all model parameters across ranks:

$$\theta_r^{(t)} \leftarrow \frac{1}{N} \sum_{r'=0}^{N-1} \theta_{r'}^{(t)}, \quad \forall r. \quad (3)$$

120 By SWA theory [Izmailov et al., 2018], averaged solutions from diverse trajectories occupy flatter
 121 minima and generalize better than any single trajectory. HDET produces this diversity explicitly
 122 in parallel rather than sequentially. The added communication cost over standard DDP is $O(1/T)$,
 123 negligible for $T \geq 100$. In our production experiments (8 H100 GPUs, $T=1,000$), this extra
 124 AllReduce adds approximately 0.1% wall-clock time.

125 3.4 Automatic Learning Rate Controller

126 After S_{warm} warmup steps, at each sync step k the controller computes per-rank losses $L_r^{(k)} =$
 127 $\frac{1}{T} \sum_t \ell(\theta_r^{(t)}, \mathcal{B}_r^{(t)})$, normalizes them against the mean $\bar{L}^{(k)}$ with temperature σ , and softmax-weights
 128 each rank:

$$s_r^{(k)} = \frac{\bar{L}^{(k)} - L_r^{(k)}}{\sigma}, \quad w_r^{(k)} = \frac{\exp(s_r^{(k)})}{\sum_{r'} \exp(s_{r'}^{(k)})}. \quad (4)$$

129 The deviation of the performance-weighted average LR from the unweighted mean is the *hypergradi-*
 130 *ent* signal:

$$\tilde{\eta}^{(k)} = \sum_r w_r^{(k)} \eta_r^{(kT)}, \quad \bar{\eta}^{(k)} = \frac{1}{N} \sum_r \eta_r^{(kT)}, \quad (5)$$

$$\Delta^{(k)} = \tilde{\eta}^{(k)} - \bar{\eta}^{(k)}. \quad (6)$$

131 $\Delta > 0$ when higher-LR replicas outperform, signaling the base LR should rise. A momentum velocity
 132 smooths the update:

$$v^{(k+1)} = \beta v^{(k)} + (1 - \beta) \Delta^{(k)}, \quad \bar{\eta}^{(k+1)} = \bar{\eta}^{(k)} (1 - \gamma) + v^{(k+1)} \lambda. \quad (7)$$

133 where γ is set to match OneCycleLR’s decay floor over $K=(S-S_{\text{warm}})/T$ remaining sync intervals:

$$\gamma = 1 - \exp\left(-\frac{\log(f_{\text{div}}/f_{\text{final}})}{K}\right). \quad (8)$$

134 **Random reassignment across hyperparameters.** After updating $\bar{\eta}^{(k+1)}$, a fresh random permuta-
 135 tion π re-assigns multipliers to ranks:

$$\eta_r^{(k+1)} = \bar{\eta}^{(k+1)} \cdot \rho_{\pi(r)}. \quad (9)$$

136 This is critical when exploring *multiple* hyperparameters simultaneously (e.g. per-group LR’s). Fixed
 137 rank-to-value assignments would create spurious cross-parameter correlations that confound the
 138 hypergradient signal; drawing an independent permutation $\pi^{(j)}$ per hyperparameter j decorrelates
 139 them, allowing the controller to track whichever hyperparameter most explains rank-loss variance at
 140 each sync. In this paper we demonstrate simultaneous exploration across four optimizer parameter
 141 groups (embedding, no_decay, non_embedding, transformer), whose emergent differential
 142 decay rates are analyzed in §5.

Algorithm 1 Hyperparameter-Divergent Ensemble Training (HDET)

Require: Model θ , optimizer, total steps S , world size N , OneCycleLR params $(\eta_{\max}, f_{\text{div}}, f_{\text{final}}, p)$, spread ratio α , sync interval T , warmup steps S_{warm} , auto-LR params (β, σ, λ)

```
1: Init:  $v \leftarrow \mathbf{0}$ ;  $L_{\text{acc}} \leftarrow 0$ 
2: Set  $\rho_r$  via Eq. (1); scale  $\text{base\_lrs} \leftarrow \text{base\_lrs} \times \rho_r$  on rank  $r$ 
3: for  $t = 1, \dots, S$  do
4:   Forward + backward; AllReduce gradients  $\bar{g}^{(t)}$  (standard DDP)
5:   Compute  $\eta_r^{(t)}$  from OneCycleLR formula with scaled  $\text{base\_lrs}$ 
6:   Update:  $\theta_r \leftarrow \theta_r - \eta_r^{(t)} \cdot \bar{g}^{(t)}$ 
7:    $L_{\text{acc}} \leftarrow L_{\text{acc}} + \ell(\theta_r, \mathcal{B}_r^{(t)})$ 
8:   if  $t \bmod T = 0$  then
9:     Converge: AllReduce  $\theta_r \leftarrow \frac{1}{N} \sum_{r'} \theta_{r'}$  (Eq. 3)
10:    if auto-LR enabled and  $t \geq S_{\text{warm}}$  then
11:       $L_r \leftarrow L_{\text{acc}}/T$ ;  $L_{\text{acc}} \leftarrow 0$ 
12:      AllGather  $\{L_r\}_{r=0}^{N-1}$ ; compute  $\{w_r\}$  via Eq. (4)
13:      AllGather  $\{\eta_r^{(t)}\}_{r=0}^{N-1}$ ; compute  $\tilde{\eta}, \bar{\eta}, \Delta$  via Eqs. (5)–(6)
14:       $v \leftarrow \beta v + (1 - \beta)\Delta$  (Eq. 7)
15:       $\bar{\eta} \leftarrow \bar{\eta}(1 - \gamma) + v \cdot \lambda$  (Eq. 7)
16:      Sample permutation  $\pi$  for random hyperparameter combinations; set  $\eta_r \leftarrow \bar{\eta} \cdot \rho_{\pi(r)}$ 
17:    else
18:      Update  $\eta_r^{(t)}$  using standard OneCycleLR formula
19:    end if
20:  end if
21: end for
22: return  $\theta_r$  (all ranks identical at last sync step)
```

143 3.5 Warm Noisy Initialization

144 HDET can optionally be seeded from a pre-trained checkpoint rather than from random initialization.
145 Before the first fan-out phase, independent Gaussian noise is added to each parameter:

$$\theta_r \leftarrow \bar{\theta}_0 + \epsilon_r, \quad \epsilon_r \sim \mathcal{N}\left(0, \frac{\nu^2 \|\bar{\theta}_0\|^2}{d}\right), \quad (10)$$

146 where $\bar{\theta}_0$ is the pre-trained checkpoint, d is the total number of parameters, and ν controls the
147 perturbation scale (we use $\nu=0.01$ throughout). This initializes each replica in a distinct neighborhood
148 of a high-quality solution, providing a warm start that accelerates early convergence and encourages
149 replicas to explore different regions of the loss landscape near the checkpoint.

150 Warm initialization *alone* is insufficient to stabilize training at aggressive learning rates: without
151 periodic weight averaging, noisy weights compound with high-LR updates and eventually diverge
152 (the **Warm-Init** ablation in §4 confirms this). Its role is to supply better-quality starting points for
153 the ensemble, not to replace the stabilization provided by the fan-out/converge cycle. When HDET
154 weight averaging is active, warm-init and the auto-LR controller each contribute independently, as
155 the ablation results demonstrate.

156 **Remark 1** (Generalization to arbitrary hyperparameters). *Algorithm 1 is presented for learning*
157 *rate η but applies to any scalar hyperparameter h satisfying: (i) varying h across replicas does*
158 *not change model architecture (so AllReduce averaging remains well-defined), and (ii) h influences*
159 *training loss differentially (so loss comparisons carry signal about h 's optimal value). Candidates*
160 *include dropout rate, label-smoothing coefficient, weight decay, and attention temperature. In each*
161 *case $\Delta = \bar{h} - \tilde{h}$ is a zero-order hypergradient estimating $-\partial\mathcal{L}/\partial h$ from inter-replica loss differences,*
162 *without back-propagating through the training loop. Multiple hyperparameters can be co-explored*
163 *simultaneously by assigning each replica an independent spread vector, with a separate random*
164 *permutation per hyperparameter to prevent cross-parameter correlation.*

Table 1: Training loss on production news-feed data. Lower is better. † indicates training crash (loss grew unboundedly). All HDET variants use $\eta_{\max}=0.0009$, which is $9\times$ higher than the conservative baseline.

Model	Description	Avg LR	Train Loss
Baseline-Low	Standard DDP, $\alpha=0$	0.0001	3.294
Baseline-High	Standard DDP, $\alpha=0$	0.0009	4.169†
Warm-Init	Perturbed init, $\alpha=0$	0.0009	4.674†
HDET w/o auto-LR	LR spread + warm init, no auto-LR	0.0009	3.280
HDET w/o warm init	LR spread + auto-LR, cold start	0.0009	3.281
HDET (full)	LR spread + warm init + auto-LR	0.0009	3.277

4 Experiments

4.1 Experimental Setup

Task and data. We evaluate on a production-scale news-feed dataset from a major social network, comprising one year of user–item interaction logs. Models are trained jointly on three engagement tasks (**Contribution, Like, LongDwell**) for **one epoch**—standard in production settings where the data stream is non-repeating and any change in training loss directly reflects optimization quality.

Backbone. All experiments share the same fixed **AttnMVP** backbone [Cheng, 2026] with identical architecture, optimizer (Adam), and feature set. Backbone details are orthogonal to HDET and are reported in Cheng [2026]; our focus is solely on the training dynamics.

HDET configuration. All variants run on **8 NVIDIA H100 GPUs** ($N=8$); each single-epoch run takes ≈ 30 hours, constraining ablation breadth. Spread $\alpha=1/9$, giving multipliers spanning $[0.0008, 0.001]$ with mean 0.0009 (matching the baseline). $T=1,000$ steps; $S_{\text{warm}}=100,000$ steps; auto-LR: $\beta=0.9, \sigma=0.1, \lambda=0.5$ (fixed, not tuned).

Ablation configurations. We compare six configurations that isolate each HDET component:

1. **Baseline-Low** ($\eta=0.0001$, no spread): conservative-LR reference.
2. **Baseline-High** ($\eta=0.0009$, no spread): high-LR without ensemble stabilization.
3. **Warm-Init** ($\eta=0.0009$, no spread): warm noisy initialization from a pre-trained checkpoint (§3.5), no weight averaging. Tests whether initialization alone is sufficient for high-LR stability.
4. **HDET w/o auto-LR** ($\eta=0.0009$, spread + warm init): fan-out/converge with LR spread and weight averaging; auto-LR disabled.
5. **HDET w/o warm init** ($\eta=0.0009$, spread + auto-LR): full HDET from a cold start; ablates warm initialization.
6. **HDET (full)** ($\eta=0.0009$, spread + warm init + auto-LR): complete method.

We report average training loss across the three tasks (lower is better); † denotes training crash.

4.2 Main Results

4.3 Analysis

Stability (Figure 2). Both Baseline-High and Warm-Init diverge at $\eta=0.0009$; HDET trains stably throughout. Warm-Init diverges *more* severely than Baseline-High (4.674 vs. 4.169), confirming that weight averaging—not initialization—is the primary stabilizer.

Quality and ablation. HDET w/o auto-LR (3.280) already beats Baseline-Low (3.294) at $9\times$ higher LR, showing that LR spread + weight averaging alone discovers better optima. The two ablations—w/o auto-LR (3.280) and w/o warm init (3.281)—each contribute independently and

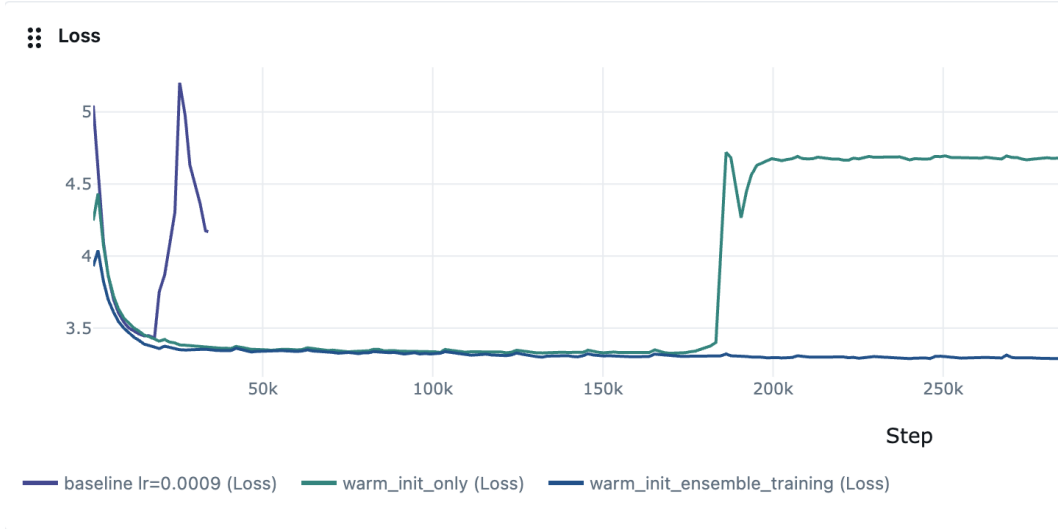


Figure 2: Training loss vs. steps at $\eta_{\max}=0.0009$. **Baseline** (gray) crashes at $\approx 18\text{K}$ steps, stabilizing at 4.169. **Warm-Init** (teal) survives to $\approx 185\text{K}$ steps then diverges (4.674): warm initialization extends stability but cannot replace weight averaging. **HDET** (dark blue) converges smoothly to 3.277—below Baseline-Low (3.294, not shown) trained at $9\times$ lower LR.

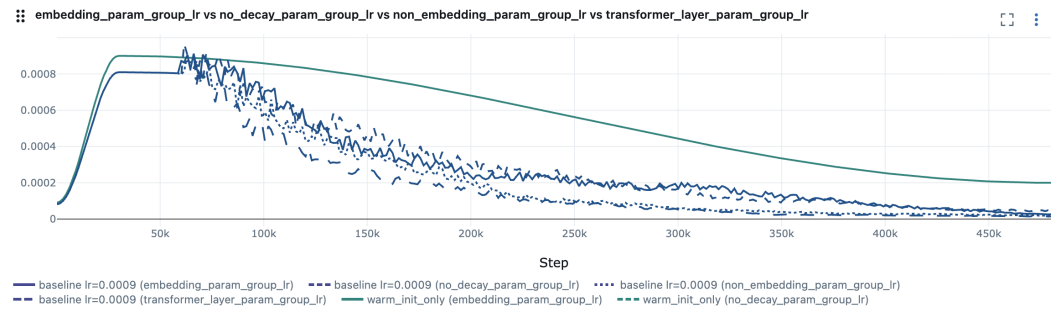


Figure 3: Per-group LR schedules for four parameter groups. Solid: baseline (OneCycleLR, $\eta_{\max}=0.0009$). Dashed: HDET GPU 0 (lowest-LR replica, $\rho_0 \cdot 0.0009=0.0008$); the 8-replica mean equals the 0.0009 baseline. Periodic fluctuations mark each sync interval where the auto-LR controller updates the base schedule.

comparably (≈ 0.013 – 0.014 each), and HDET (full, 3.277) combines both for the best result. The auto-LR gain is modest here because the initial schedule is already near-optimal; larger gains are expected when the hand-tuned baseline is farther from optimal.

Statistical reliability. We repeated training over **10+ independent models**; HDET consistently outperformed Baseline-Low on offline AUC and NE across all replications. The model is deployed in the production ranking system.

Per-group LR schedules (Figure 3). Figure 3 shows GPU 0 (peak LR $0.0008 = \rho_0 \cdot 0.0009$, the lowest-LR replica) for clarity; the 8-replica mean equals the 0.0009 baseline. The auto-LR controller produces periodic fluctuations at each sync and drives all groups to a residual LR of ≈ 0.0002 rather than near-zero, reflecting its discovery that a non-trivial final LR benefits this task.

5 Discussion

Exploration-exploitation in hyperparameter space. HDET implements a principled exploration-exploitation trade-off directly in hyperparameter space. The *fan-out* phase is the exploration step: replicas diverge under a structured spread of learning rates, simultaneously probing distinct regions of the loss landscape that no single schedule could visit concurrently. The *converge* phase is the exploitation step: AllReduce averaging collapses the ensemble into a single model that harvests the best of all explored trajectories, re-anchoring parameters near the consensus of the full replica population. Because the LR spread always keeps some replicas conservative, this consensus is guaranteed to lie in a stable region—which is why periodic averaging prevents divergence where a single aggressive LR cannot. Warm-Init without HDET illustrates the same mechanism in reverse: with no converge phase to re-anchor parameters, high-LR updates on perturbed weights compound unchecked until irrecoverable divergence at $\sim 185\text{K}$ steps.

Emergent differential LR decay. The auto-LR controller discovers distinct per-group decay rates without manual specification (Figure 3): `transformer` decays fastest (dense high-variance gradients converge quickly), `non_decay` parameters slowest (flat landscapes absorb gradient signal throughout), with embeddings and non-embedding layers in between. This ordering mirrors discriminative fine-tuning [Howard and Ruder, 2018] but emerges *automatically* from cross-replica loss signals.

Limitations. HDET’s auto-LR adapts per-group schedules from model performance across replicas, complementing Adam’s [Kingma and Ba, 2014] per-parameter gradient-moment adaptation; combining them is a natural next step. The three additional hyperparameters (α , T , σ) were fixed without tuning and consistently outperformed the baseline, though their interaction and architecture-dependence warrant further study.

6 Conclusion and Future Work

HDET converts the wasted uniformity of DDP replicas into a structured LR exploration ensemble via a fan-out/converge cycle ($O(1/T)$ extra communication), with an auto-LR controller that adapts the base schedule online from cross-replica loss signals and warm noisy initialization that seeds training near a high-quality checkpoint. On a production recommendation model, HDET trains stably with much higher LR than the LR that causes divergence in standard DDP and achieves a lower final loss (3.277 vs. 3.294), while its auto-LR controller autonomously discovers the empirically correct per-group decay ordering (transformer fastest, non-decay slowest).

Future directions. (i) *General hyperparameter exploration.* As formalized in Remark 1, any scalar hyperparameter that does not change model architecture (dropout, weight decay, temperature) can be substituted for LR, turning the cluster into a zero-cost hyperparameter search engine using inter-replica losses as hypergradients. (ii) *Group-aware initialization.* The emergent decay ordering provides an empirical prior for initializing per-group spread ratios, potentially accelerating group-wise schedule discovery. (iii) *LLM pretraining.* Extending HDET to multi-epoch pretraining over hundreds of billions of tokens offers more sync intervals for the auto-LR to adapt, with warm-init generalized to early-stage cold starts. (iv) *Convergence theory.* A formal analysis characterizing the interplay of LR spread, sync interval, and averaging on convergence and implicit regularization remains an open question.

References

References

- A. G. Baydin, R. Cornish, D. M. Rubio, M. Schmidt, and F. Wood. Online learning rate adaptation with hypergradient descent. In *International Conference on Learning Representations (ICLR)*, 2018.
- J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, 2011.

255 L. Blier, P. Wolinski, and Y. Ollivier. Learning with random learning rates. *arXiv preprint*
256 *arXiv:1810.01322*, 2019.

257 H. Cheng. Beyond interleaving: Causal attention reformulations for generative recommender systems,
258 2026. URL <https://arxiv.org/abs/2603.10369>.

259 A. Defazio and K. Mishchenko. Learning-rate-free learning by D-adaptation. In *International*
260 *Conference on Machine Learning (ICML)*, 2023.

261 A. Defazio, X. A. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky. The road less
262 scheduled. *arXiv preprint arXiv:2405.15682*, 2024.

263 T. Dimlioglu and A. Choromanska. GRAWA: Gradient-based weighted averaging for distributed
264 training of deep learning models. *arXiv preprint arXiv:2403.04206*, 2024.

265 A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Szegedy, A. Sessa,
266 A. Galashov, and A. T. Cemgil. DiLoCo: Distributed low-communication training of language
267 models. *arXiv preprint arXiv:2311.08105*, 2023.

268 S. Fort, H. Hu, and B. Lakshminarayanan. Deep ensembles: A loss landscape perspective. In *arXiv*
269 *preprint arXiv:1912.02757*, 2019.

270 V. Gupta, S. A. Serrano, and D. DeCoste. Stochastic weight averaging in parallel: Large-batch
271 training that generalizes well. In *International Conference on Learning Representations (ICLR)*,
272 2020.

273 F. Haddadpour, M. M. Kamani, M. Mahdavi, and V. R. Cadambe. Local SGD with periodic averaging:
274 Tighter analysis and adaptive synchronization. In *Advances in Neural Information Processing*
275 *Systems (NeurIPS)*, volume 32, 2019.

276 J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural*
277 *Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

278 J. Howard and S. Ruder. Universal language model fine-tuning for text classification. In *Proceedings*
279 *of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 328–339,
280 2018.

281 M. Ivgi, O. Hinder, and Y. Carmon. DoG is SGD’s best friend: A parameter-free dynamic step size
282 schedule. *arXiv preprint arXiv:2302.12022*, 2023.

283 P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson. Averaging weights leads to
284 wider optima and better generalization. In *Uncertainty in Artificial Intelligence (UAI)*, 2018.

285 M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals,
286 T. Green, I. Dunning, K. Simonyan, et al. Population based training of neural networks. In *arXiv*
287 *preprint arXiv:1711.09846*, 2017.

288 D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*,
289 2014.

290 B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty esti-
291 mation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*,
292 volume 30, 2017.

293 S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan,
294 P. Damania, et al. PyTorch distributed: Experiences on accelerating data parallel training. In
295 *Proceedings of the VLDB Endowment*, volume 13, pages 3005–3018, 2020.

296 I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International*
297 *Conference on Learning Representations (ICLR)*, 2017.

298 A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein,
299 L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In
300 *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

301 M. Rolinek and G. Martius. L4: Practical loss-based stepsize adaptation for deep learning. In
302 *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

303 T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative
304 to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

305 L. N. Smith. Cyclical learning rates for training neural networks. In *2017 IEEE Winter Conference*
306 *on Applications of Computer Vision (WACV)*, pages 464–472. IEEE, 2017.

307 L. N. Smith and N. Topin. Super-convergence: Very fast training of neural networks using large
308 learning rates. *Proceedings of SPIE*, 11006:1100612, 2019.

309 J. Snoek, H. Larochelle, and R. P. Adams. Practical bayesian optimization of machine learning
310 algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.

311 A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin.
312 Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

313 D. Wierstra, T. Forster, J. Peters, and J. Schmidhuber. Natural evolution strategies. In *Journal of*
314 *Machine Learning Research*, volume 15, pages 949–980, 2014.

315 M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong,
316 A. Farhadi, Y. Carlin, S. Kornblith, et al. Model soups: averaging weights of multiple fine-tuned
317 models improves accuracy without increasing inference time. In *International Conference on*
318 *Machine Learning (ICML)*, pages 23965–23998, 2022.

319 S. Zhang, A. E. Choromanska, and Y. LeCun. Deep learning with elastic averaging SGD. In *Advances*
320 *in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state four claims: (1) HDET repurposes DDP replicas for LR exploration at zero hardware overhead; (2) the auto-LR controller adapts the base schedule online from cross-replica loss signals; (3) warm noisy initialization combined with ensemble training maximizes stability and quality; (4) HDET trains stably at $9\times$ the LR that causes divergence in standard DDP and achieves lower final loss. All four are directly supported by the ablation results in Table 1 and Figures 2–3.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 (Limitations paragraph) discusses: (1) the three additional hyperparameters (α , T , σ) were fixed without per-run tuning and their interaction warrants further study; (2) the emergent per-group LR decay ordering may vary across architectures beyond AttnMVP; (3) evaluation is on a single production dataset, limiting generalizability claims. Section 6 further notes that convergence theory for the fan-out/converge cycle remains an open question.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper makes no formal theoretical claims requiring proof. Remark 1 (generalization to arbitrary hyperparameters) and the connections to SWA and NES in Section 2 are informal observations rather than theorems. A formal convergence analysis is listed as future work in Section 6.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 fully specifies the HDET configuration: $N=8$, $\alpha=1/9$, $T=1,000$, $S_{\text{warm}}=100,000$, $\beta=0.9$, $\sigma=0.1$, $\lambda=0.5$, $\nu=0.01$ for warm init. Algorithm 1 provides a complete pseudocode description. A reference PyTorch implementation is released at https://github.com/hailingc/ensemble_training. The production dataset is proprietary, but the method description is self-contained and applies to any DDP+OneCycleLR training setup.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The training dataset (one year of LinkedIn news-feed interaction logs) is proprietary and cannot be released. The HDET scheduler implementation is open-sourced at https://github.com/hailingc/ensemble_training as a drop-in PyTorch scheduler, allowing practitioners to apply the method to their own datasets.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.

- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution.
- The instructions should contain the exact command and environment needed to run to reproduce the results.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4 specifies hardware (8 H100 GPUs), optimizer (Adam), all HDET hyperparameters, the six ablation configurations, and the evaluation metric (average training loss across three tasks). The auto-LR parameters β , σ , λ and warm-init scale ν are all reported as fixed values used without per-run tuning.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Section 4 (Statistical reliability paragraph) reports that training was repeated for over 10 independent models across different random seeds and data orderings. HDET consistently outperformed Baseline-Low on both offline AUC and Normalized Entropy (NE) across all replications. Individual run costs (≈ 30 hours on 8 H100 GPUs) preclude reporting confidence intervals in the main table, but the 10+ replication protocol provides the statistical basis for the reported claims.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (e.g., train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4 reports that all experiments use 8 NVIDIA H100 GPUs and that each single-epoch training run takes approximately 30 hours. Section 3.3 reports that the extra weight AllReduce adds approximately 0.3% wall-clock time over standard DDP at $T=1,000$.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This work proposes a training methodology improvement for data-parallel deep learning. Experiments use production interaction logs processed under standard industry data-access controls; no personal data is released or described. The method itself introduces no new high-risk capabilities.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: HDET reduces the need for expensive hyperparameter sweeps, lowering the compute cost of training large models and making high-quality training more accessible. The method is architecture-agnostic and applicable to any DDP-based training pipeline, including LLM pretraining (discussed in Section 6). No negative societal impacts are anticipated: HDET is a training efficiency improvement with no novel data collection, model deployment, or inference-time changes.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations, privacy considerations, and security considerations.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: HDET is a training scheduler and synchronization protocol. It does not release a pre-trained model, dataset, or any artifact with elevated misuse risk.

Guidelines:

528 • The answer [N/A] means that the paper poses no such risks.
529 • Released models that have a high risk for misuse or dual-use should be released with
530 necessary safeguards to allow for controlled use of the model.

531 **12. Licenses for existing assets**

532 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
533 the paper, properly credited and are the license and terms of use explicitly mentioned and
534 properly respected?

535 Answer: [Yes]

536 Justification: PyTorch and its OneCyclicLR scheduler [Paszke et al., 2019] are cited; HDET
537 subclasses the scheduler under PyTorch’s BSD license. All baseline methods and related
538 work are properly cited. No third-party datasets or model weights are redistributed.

539 Guidelines:

540 • The answer [N/A] means that the paper does not use existing assets.
541 • The authors should cite the original paper that produced the code package or dataset.
542 • The authors should state which version of the asset is used and, if possible, include a
543 URL.
544 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.

545 **13. New assets**

546 Question: Are new assets introduced in the paper well documented and is the documentation
547 provided alongside the assets?

548 Answer: [Yes]

549 Justification: The HDET scheduler is released at [https://github.com/hailingc/](https://github.com/hailingc/ensemble_training)
550 ensemble_training with usage instructions. Algorithm 1 serves as the authoritative
551 pseudocode reference.

552 Guidelines:

553 • The answer [N/A] means that the paper does not release new assets.
554 • Researchers should communicate the details of the dataset/code/model as part of their
555 submissions via structured templates. This includes details about training, license,
556 limitations, etc.

557 **14. Crowdsourcing and research with human subjects**

558 Question: For crowdsourcing experiments and research with human subjects, does the paper
559 include the full text of instructions given to participants and screenshots, if applicable, as
560 well as details about compensation (if any)?

561 Answer: [N/A]

562 Justification: The paper does not involve crowdsourcing or human subjects experiments.

563 Guidelines:

564 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
565 with human subjects.
566 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
567 or other labor should be paid at least the minimum wage in the country of the data
568 collector.

569 **15. Institutional review board (IRB) approvals or equivalent for research with human**
570 **subjects**

571 Question: Does the paper describe potential risks incurred by study participants, whether
572 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
573 approvals (or an equivalent approval/review based on the requirements of your country or
574 institution) were obtained?

575 Answer: [N/A]

576 Justification: No human subjects were involved in this research.

577 Guidelines:

- 578 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
579 with human subjects.
580 • Depending on the country in which research is conducted, IRB approval (or equivalent)
581 may be required for any human subjects research. If you obtained IRB approval, you
582 should clearly state this in the paper.

583 **16. Declaration of LLM usage**

584 Question: Does the paper describe the usage of LLMs if it is an important, original, or
585 non-standard component of the core methods in this research? Note that if the LLM is used
586 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
587 scientific rigor, or originality of the research, declaration is not required.

588 Answer: [N/A]

589 Justification: LLM pretraining is discussed as a future application direction in Section 6,
590 not as a component of the methodology. No LLM was used as part of the core method
591 development or evaluation.

592 Guidelines:

- 593 • The answer [N/A] means that the core method development in this research does not
594 involve LLMs as any important, original, or non-standard components.
595 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
596 be described.